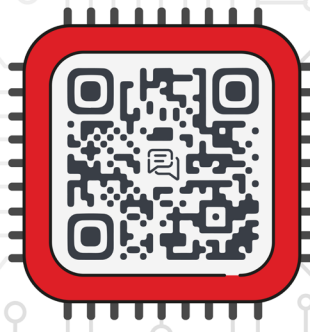




3



9



1350 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

الوراثة والبنائي

د. فراس ضعيف

د. معتز النجرس



البرمجة 2

RB Informatcs; 26/04/2026

البواني Constructors

عند إنشاء كائن من الصنف بالشكل الآتي:

```
Student s = new Student();
```

فإن العملية السابقة تعني إنشاء كائن من الصنف Student والتعامل معه من خلال المتغير s.

إذا كانت هناك حاجة إلى تحديد الطريقة التي يُنشأ بها الكائن، فإن ذلك يتم من خلال إنشاء Constructor داخل الصنف.

فالباني يحدد آلية بناء الكائن، أي المنطق البرمجي الذي يُنفَّذ أثناء إنشاء الكائن.

الباني Constructor هو جزء خاص داخل الصنف يشبه الدالة من حيث احتوائه على تعليمات برمجية، لكنه ليس Method عادية. ويتم استدعاؤه تلقائياً عند إنشاء كائن باستخدام الكلمة المفتاحية new.

كذلك يُستخدم الـ constructor لتنفيذ تعليمات معينة عند إنشاء الكائن، وغالباً لاستقبال قيم أولية مثل name و age.

تُسمّى الخصائص الموجودة داخل الصنف Properties، مثل: name, age.

أما الدوال الموجودة داخل الصنف فتُسمّى Methods. وبذلك فإن الـ method هي في الأصل function، لكنها تُسمّى method لأنها مكتوبة ضمن صنف.

Default Constructor

يوجد نوع من البواني يُسمّى Default Constructor، وهو الباني الافتراضي الذي يسمح بإنشاء كائن من الصنف دون تمرير أي معاملات.

```
class Person {
    String name;
    int age;
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
    }
}
```

في المثال السابق يمكن إنشاء كائن من الصنف Person دون تمرير أي معاملات؛ لأن Java توفر بانياً افتراضياً في حال عدم تعريف أي بانٍ آخر داخل الصنف.

أما إذا تمت كتابة بانٍ يستقبل معاملات، ونريد السماح في الوقت نفسه بإنشاء كائن دون معاملات، فيجب إعادة كتابة البانٍ الافتراضي يدوياً. وبذلك يصبح بالإمكان استخدام بانٍ يستقبل معاملات، وبانٍ آخر لا يستقبل أي معاملات. تسمى هذه الحالة Overloading أي وجود أكثر من دالة أو بانٍ بالاسم نفسه، مع اختلاف في عدد أو نوع المعاملات. مثال توضيحي على Overloading

```
class Person {
    String name;
    int age;
    Person() {
        name = "Unknown";
        age = 0;
    }
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
class Main {
    public static void main(String[] args) {
        Person p1 = new Person();
        Person p2 = new Person("Ahmad", 20);
    }
}
```

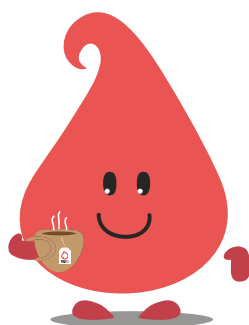
في هذا المثال يوجد بانين داخل الصنف Person:
الأول لا يستقبل أي معاملات:

```
Person(){}
```

والثاني يستقبل الاسم والعمر:

```
Person(String name, int age) {
    this.name = name;
    this.age = age;
}
```

وهذا يتيح إنشاء كائن بطريقتين مختلفتين.



لنفترض وجود صنف باسم Student يحتوي على باني يستقبل الاسم والعمر:

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        name = name;
        age = age;
    }
}
```

في هذه الحالة يحدث تشابه في الأسماء بين معاملات الباني والخصائص الموجودة داخل الصنف. في العبارة `name = name` يُفسر الاسم في الطرفين على أنه المعامل المحلي داخل الباني، لذلك لا يتم تعديل خاصية الصنف.

لذلك تستخدم الكلمة المفتاحية `this` للإشارة إلى الكائن الحالي، ومن خلالها يمكن الوصول إلى خصائص الصنف عند تشابه أسمائها مع أسماء المعاملات.

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

في هذه الحالة: `this.name` تشير إلى الخاصية `name` الموجودة داخل الصنف، بينما `name` تشير إلى المعامل المحلي الذي استقبله الباني.

بمعنى آخر، الكلمة `this` تشير إلى الكائن الحالي المنشأ من الصنف، وتستخدم للتمييز بين خصائص الصنف والمتغيرات المحلية عند تشابه الأسماء.

استخدامات أخرى للكلمة this

يمكن استخدام `this` أيضاً لاستدعاء دالة موجودة داخل الصنف نفسه، لكن استخدامه هنا اختياري غالباً، لأن الدالة يمكن استدعاؤها مباشرة من داخل الصنف. لنفترض وجود الدالة الآتية:

```
void sayHi() {
    System.out.println(this.name);
}
```

إذا كان المطلوب طباعة اسم الكائن بعد إنشائه مباشرة، يمكن استدعاء الدالة `sayHi` داخل الباني:

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
        this.sayHi();
    }

    void sayHi() {
        System.out.println(this.name);
    }
}
```

يمكن أيضاً كتابة sayHi() فقط، لأننا نستدعي دالة من داخل الصنف نفسه، لكن استخدام this.sayHi() يوضح أن الدالة تابعة للكائن الحالي.

عند إنشاء كائن من الصنف Person، ينفذ الباني أولاً، ثم يستدعي الدالة: sayHi() المكتوبة بالشكل this.sayHi() فتنفذ التعليمة System.out.println(this.name) وبذلك تُطبع قيمة الخاصية name الخاصة بالكائن الحالي.

استدعاء Constructor من Constructor آخر

من استخدامات this أيضاً استدعاء باني من باني آخر داخل الصنف نفسه باستخدام الصيغة this(...). فلا يتم استدعاء الباني باسمه مباشرة، وإنما باستخدام this(...) مع تمرير القيم المناسبة للباني المطلوب كما يلي: this(name, age) تسمى هذه الفكرة: Constructor Chaining أي سلسلة البواني، أو حلقة استدعاء البواني داخل الصنف نفسه.

```
class Person {
    String name;
    int age;

    Person() {
        this("Unknown", 0);
    }

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

في المثال السابق، عند استدعاء الباني الذي لا يستقبل معاملات: Person() فإنه يستدعي الباني الآخر الذي يستقبل name و age من خلال: this("Unknown", 0) وهكذا يتم تقليل التكرار داخل البواني.

يجب أن تكون التعليمة this(...) أول تعليمة داخل الباني عند استخدامها لاستدعاء باني آخر.

مشكلة التكرار بين الأصناف

لنفترض وجود صنف باسم Person يحتوي على الخاصية name ودالة تطبع رسالة تعريفية:

```
class Person {
    String name;

    void sayHi() {
        System.out.println("Hi I am " + this.name);
    }
}
```

ثم يوجد صنف آخر باسم Teacher يحتوي أيضًا على name و college ودالة sayHi:

```
class Teacher {
    String name;
    String college;

    void sayHi() {
        System.out.println("Hi I am " + this.name);
    }
}
```

وكذلك يوجد صنف ثالث باسم Student يحتوي على name ودالة sayHi، بالإضافة إلى دالة أخرى:

```
class Student {
    String name;

    void sayHi() {
        System.out.println("Hi I am " + this.name);
    }

    void doExam() {
        System.out.println(this.name + " is doing an exam");
    }
}
```

المشكلة في هذا المثال هي وجود تكرار واضح، إذ إن الخاصية name والدالة sayHi مكررتان في أكثر من صنف.

في Java تعرف الدوال عادة داخل أصناف، وتكون مرتبطة بالصنف أو بالكائنات المنشأة، فلا يمكن استدعاء الدالة عادةً دون إنشاء كائن من الصنف.

حتى داخل الدالة main، لا يمكن استدعاء دالة عادية تابعة لصنف ما دون إنشاء كائن من ذلك الصنف، ما لم تكن الدالة معرفة بطريقة خاصة مثل static.

Inheritance الوراثة

تستخدم الوراثة لتقليل التكرار بين الأصناف وإعادة استخدام الخصائص والدوال المشتركة بينها. فإذا وُجدت خصائص أو دوال مشتركة بين أكثر من صنف، يمكن وضعها داخل صنف عام، ثم اشتقاق الأصناف الأخرى منه

العلاقات بين الأصناف

علاقة الوراثة

عند ملاحظة أن صنفًا معيّنًا يحقق جميع خصائص ودوال صنف آخر، ويضيف عليها خصائص أو دوالاً جديدة، يمكن اشتقاقه من ذلك الصنف.

فمثلاً: كل طالب هو شخص.

أي إن الصنف Student يمكن اعتباره نسخة موسّعة من الصنف Person. لذلك يمكن اشتقاق Student من Person. تتم كتابة ذلك بالشكل الآتي:

```
class Student extends Person {
}
```

بهذه الطريقة، تنتقل العناصر المشتركة من Person إلى Student بحسب محددات الوصول، ويكتب داخل Student فقط ما يمثل الإضافات الخاصة بالطالب.

مفهوم Instance

الـ instance هو الكائن الفعلي الموجود في الذاكرة بعد إنشائه من صنف معيّن.

فإذا كان هناك صنف يمثل الطلاب، فإن كل طالب محدد بقيمه الخاصة يُعدّ instance من هذا الصنف.

على سبيل المثال، الطالب قد يمتلك الخصائص الآتية: id, name

وعند إعطاء هذه الخصائص قيماً محددة، يصبح هناك مثال فعلي من الطالب، أي instance.

بصيغة أخرى، الصنف يحدد الشكل العام، أما الـ instance فهو كائن حقيقي من هذا الصنف بقيم محددة.

سلسلة الوراثة

يمكن أن يشتق صنف من صنف آخر، ثم يشتق صنف ثالث من الصنف الثاني. وهذا يُعدّ جزءاً من مفهوم الوراثة. مثال عام:

```
class Person {
}
class Student extends Person {
}
class GraduateStudent extends Student {
}
```

في هذا المثال، الصنف GraduateStudent يرث من Student، وStudent يرث من Person.

في Java يمكن للصف أن يرث مباشرة من صف واحد فقط، لكن يمكن تشكيل سلسلة وراثية كما في المثال السابق.

الوراثة في Java و C++

في لغة C++ يمكن للصف أن يشتق من أكثر من صف واحد، أي يمكن تحقيق الوراثة المتعددة:

```
class C : public A, public B{
};
```

أما في Java، فإن الصف لا يمكنه أن يرث مباشرةً من أكثر من صف واحد باستخدام extends.

يمكن تعويض جزء من فكرة الوراثة المتعددة باستخدام ال Interfaces التي سنمر عليها لاحقاً.

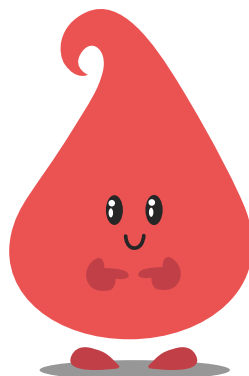
إعادة تعريف الدوال Override

عند وجود دالة في الصف الأب، ثم إعادة كتابتها داخل الصف الابن بالاسم نفسه ونفس المعاملات، فإن هذه العملية تسمى Override

وتعدّ هذه الفكرة أحد مفاهيم تعدد الأشكال: Polymorphism
مثال:

```
class Person {
    void sayHi() {
        System.out.println("Hi from Person");
    }
}
class Student extends Person {
    @Override
    void sayHi() {
        System.out.println("Hi from Student");
    }
}
```

في هذا المثال، الدالة sayHi موجودة في الصف الأب Person، ثم أعيد تعريفها داخل الصف الابن Student.



Super

تستخدم الكلمة المفتاحية super داخل الصنف الابن للوصول لعناصر الصنف الأب سواء كانت بواني أو خصائص أو دوال

استخدامات super

1. استدعاء باني الصنف الأب

تستخدم super(...) داخل باني الصنف الابن لاستدعاء باني الصنف الأب وتميرير القيم المناسبة إليه.

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

class Student extends Person {
    Student(String name) {
        super(name);
    }
}
```

في المثال السابق، تستدعي التعليمة super(name) باني الصنف الأب Person، وتمرر إليه القيمة name.

يجب أن تكون super(...) أول تعليمة داخل باني الصنف الابن عند استخدامها لاستدعاء باني الأب.

2. الوصول إلى متغيرات وتوابع الصنف الأب

تستخدم super أيضاً للوصول إلى متغير أو دالة موجودة في الصنف الأب، خصوصاً إذا أعاد الصنف الابن تعريف متغير أو دالة بالاسم نفسه.

مثال:

```
class Person {
    String name = "Ahmad";
}

class Student extends Person {
    String name = "Ali";

    void display() {
        System.out.println(name);
        System.out.println(super.name);
    }
}
```

Ali

Ahmad

في هذا المثال: name تطبع قيمة المتغير الموجود داخل الصنف الابن، وهي Ali
أما: super.name فتطبع قيمة المتغير الموجود داخل الصنف الأب، وهي Ahmad

الفرق بين this و super

الكلمة	الاستخدام	المثال	المعنى
this	تشير إلى الكائن الحالي	this.name	الوصول إلى الخاصية الموجودة في الصنف الحالي
super	الوصول إلى عناصر الصنف الأب	super.name	الوصول إلى الخاصية الموجودة في الصنف الأب

تشير this إلى الكائن الحالي داخل الصنف نفسه، ولذلك تُستخدم غالباً للوصول إلى خصائص ودوال الصنف الحالي.
أما super فتستخدم للوصول إلى عناصر الصنف الأب عند الحاجة، سواء كانت متغيرات أو دوالاً أو بواني.

